

Compounding On-Call

An incident-response agent for data pipelines that stops re-diagnosing the same failure. It reasons once, remembers, and gets measurably faster every time the same fault reappears.

INCIDENTS

6/6

REPLAY SPEEDUP

249×

TOKENS ON REPLAY

0

SERVICES LIVE

4

The knowledge exists. It just isn't actionable.

A column gets renamed in an upstream CDC source. Three hops downstream a mart starts producing nulls, a job fails its NOT NULL assertion, and someone on-call spends 45 minutes rediscovering a causal chain a colleague already traced last month.

The postmortem gets written. It goes in a wiki. Nobody reads it. Six weeks later a **different table** fails the **same way** and the 45 minutes are spent again.

From our own synthetic corpus — `pm_inventory_repeat.md`: “The fix was already written down. The cost was not the diagnosis, it was that nobody could find the previous diagnosis.”

A real trigger, not a simulated one

```
scripts/run_job.py --table orders          a genuine subprocess
  reads Parquet, asserts NOT NULL + row-count minimums
  fails → real stderr, exit 1
    |
    ▼
supervisor catches returncode != 0, captures stderr verbatim
    |
    ▼
signature(error_text)  strip timestamps, paths, counts, table name
    |
    ▼
muscle memory: seen this shape before?
├─ HIT → replay the captured path · 0 tokens · skip to verify
└─ MISS → lineage · cognee · hydradb · hotdata · rocketride
    |
    ▼
verify: re-run the real job, expect exit 0
    |
    ▼
learn: capture path · record outcome · write narrative back
```

Both the trigger and the verify are real processes. PASS means a subprocess exited 0 — not that the agent asserted success.

Five layers, five different questions

MUSCLE

Signature-keyed path store.
A hit is a dict lookup — which
is **why replay is zero
tokens**, not fewer.

COGNEE

Runbooks and postmortems
→ knowledge graph. **What
the org knew before we
started.**

HYDRADB

Accumulating resolution
memory. **What we have
learned since.**

HOTDATA

Live SQL over the
warehouse. **What is wrong
right now.** Never cached.

ROCKETRIDE

Chains the remediation:
quarantine → rerun → notify.
Acts.

WHY THREE MEMORY LAYERS

They answer questions on different time horizons, so action selection is a precedence chain:

a fix we have proven beats **a fix the runbooks prescribe** beats a built-in heuristic. Recall
that never changes a decision is decoration; ours decides.

Hard boundary: no Parquet-derived number is ever written to a memory layer. Null rates go stale in hours; lineage is stable for weeks.

Incident 1 — reasoning from scratch

INCIDENT 1 · job_orders_daily · table orders		COLD
1 Pipeline crashed	exit 1 — NOT NULL violated on customer_id: 8000 nulls across 20000 rows (40.0%)	
2 Fingerprint	sig_ba4a7a717fd1 · NULL_FLOOD	
3 Muscle memory	MISS — never seen this shape	
4 Recall	lineage 1 upstream change, owner @priya cognee 1 doc · hydradb 0 prior fixes	
5 Diagnose	null rate 40.0% today vs 0.0% known-good	
6 Decide + act	quarantine_partition because prescribed by runbook recall (cognee) quarantine → rerun_job → notify_owner	
7 Verify	exit 0 — resolved	
8 Learn	path captured · outcome recorded · narrative written	
SERVICE ACTIVITY		live timers
COGNEE	what did the org already write down?	18.5s
HYDRADB	what have we resolved before?	996ms
HOTDATA	what is wrong with the data right now?	723ms
JOB	run job_orders_daily as a subprocess	186ms

Every service call is announced before and after with a live counting timer, so a 18-second reasoning step reads as work in progress rather than a frozen screen.

Incident 3 — the same fault, a different table

INCIDENT 3 · job_shipments_daily · table shipments

WARM

- | | |
|--------------------|---|
| 1 Pipeline crashed | exit 1 – NOT NULL violated on customer_id |
| 2 Fingerprint | sig_ba4a7a717fd1 · NULL_FLOOD ← same signature |
| 3 Muscle memory | HIT – solved this shape on orders,
replaying with no reasoning |
| 4 Recall | skipped – replaying a known fix |
| 5 Diagnose | skipped – replaying a known fix |
| 6 Decide + act | skipped – replaying a known fix |
| 7 Verify | exit 0 – resolved |

SERVICE ACTIVITY

- | | | |
|-----|---|-------|
| JOB | run job_shipments_daily as a subprocess | 131ms |
| JOB | re-run to prove the fix held | 118ms |

WARM · 131ms · 0 tokens

This is the demo. A path captured on `orders` applies to `shipments` because the signature deliberately excludes the table name. Steps 4–6 grey out — the audience sees the expensive work *not happening*.

RESULTS

The compounding curve

#	TABLE	SIGNATURE	FAILURE MODE	PATH	MS	TOKENS	VERIFY
1	orders	sig_ba4a7a717fd1	NULL_FLOOD	COLD	31,165	1,902	PASS
2	shipments	sig_9c3d1a8ea10f	VOLUME_COLLAPSE	COLD	33,396	1,885	PASS
3	shipments	sig_ba4a7a717fd1	NULL_FLOOD	WARM	131	0	PASS
4	inventory	sig_9c3d1a8ea10f	VOLUME_COLLAPSE	WARM	147	0	PASS
5	inventory	sig_ba4a7a717fd1	NULL_FLOOD	WARM	102	0	PASS
6	orders	sig_9c3d1a8ea10f	VOLUME_COLLAPSE	WARM	137	0	PASS

COLD – reasoned

avg 32,280ms · 1,894 tokens · 2 runs

WARM – replayed

avg 129ms · 0 tokens · 4 runs

Same two signatures rotated across three tables. Replay triggers on signature match, not position — pick any table and we will inject a failure live.

What the docs said, and what was actually true

Three of four vendors behaved differently from their documentation. Each was found by running against the live API, and each is recorded in `docs/03_SDK_NOTES.md`.

HYDRADB · NO CYPHER

The spec's centrepiece was a 3-hop graph query. HydraDB v2 has no graph query language at all, so the lineage graph became ours, traversed in Python. HydraDB became the accumulator instead.

HOTDATA · SILENT 404

A load reporting `row_count=20000` still 404'd on query, because the catalog is always `default` regardless of the alias you set. Success then failure, with no obvious link.

COGNEE · LIST, NOT STRING

`datasets` as a bare string works locally and returns 422 remotely. Hosted responses also wrap per dataset, so unwrapping was needed or every excerpt was raw JSON.

ROTE · NO DOCS

Priority #1 in the fallback order with no published API. We implemented muscle memory ourselves — 90 lines, no network. The load-bearing claim cannot depend on an unverified SDK.

A test caught the one bug that would have killed the demo: the table name is embedded in real job output (`job_orders_daily`, `in orders:`), so keeping it out of the hash input was not enough. Without that fix nothing ever replays and there is no curve.

What is real, and what we would say if asked

- **All four vendor services run live.** Cognee on a hosted tenant, HydraDB accumulating real memories, hotdata computing over uploaded Parquet, RocketRide chaining three steps on staging. The UI header shows REAL or STUB every run — we cannot demo stubs by accident.
- **Muscle memory is ours, not a vendor.** ~90 lines, JSON on disk. Said plainly because it reads as discipline, not omission.
- **Token counts on any stubbed layer are estimates**, computed from real serialised context size and labelled as such in the report footer.
- **Every layer keeps a working stub behind a config flag.** Any vendor can be dropped with one flag and the demo still runs end to end at 7.4x.
- **Incident 1 is genuinely cold.** Both memory stores are cleared at start, and HydraDB's delete lag is filtered — so the first incident cannot read its own predecessors.

IN ONE LINE

The agent does not get smarter. It stops forgetting.

Same prompt, same tools, same model. The only thing that changes between incident 1 and incident 3 is what the system remembers — and that alone is worth 249x and every token.

COLD, REASONED

32.3s

WARM, REPLAYED

0.13s

TOKENS AVOIDED

7,576

VERIFIED

6/6